
CORSO DI ARCHITETTURE DEGLI ELABORATORI A.A. 2025/2026
PROF.SSA VALENTINA CASOLA

SIMULAZIONE DI CIRCUITI DIGITALI

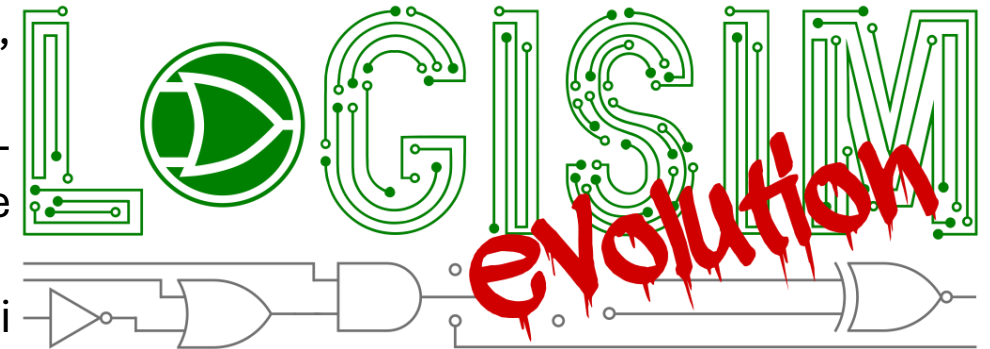
Dr. Alessandra Somma
alessandra.somma@unina.it

SIMULATORI DI CIRCUITI DIGITALI

- Un **simulatore** riproduce in **ambiente virtuale** il comportamento di un sistema reale, consentendo di osservarne il funzionamento senza doverlo fisicamente ricostruire.
 - E' possibile osservare le risposte del sistema ed comprendere i principi di funzionamento.
- I simulatori permettono di sperimentare, verificare, analizzare il comportamento di componenti o architetture, riducendo tempi, costi e rischi.
- Nell'*informatica* ed elettronica digitale, la simulazione è un passaggio essenziale per passare dal concetto di circuito logico all'implementazione fisica del circuito o di un'architettura.
- Un **simulatore di circuiti digitali** consente di progettare e testare reti logiche (combinatorie e sequenziali) in modo visuale ed interattivo. Tramite un'interfaccia grafica è possibile collegare porte logiche (AND, OR, NOT, NAND, etc.), realizzare circuiti plexer (mux/demux), codificatori, contatori, flip-flop e piccole unità aritmetico-logiche (ALU).
 - Molti simulatori permettono di visualizzare segnali temporali, simulare *clock* e verificare tabelle di verità.

PRINCIPALI SIMULATORI DI CIRCUITI DIGITALI: LOGISIM

- **Logisim Evolution** (<https://github.com/logisim-evolution/logisim-evolution>) è uno dei simulatori di circuiti digitali più diffuso in ambito accademico. Deriva dal progetto originale Logisim (<https://github.com/logisim-evolution/logisim-evolution>) di Carl Burch, ampliandone le funzionalità e mantenendo l'approccio visuale.
 - Permette di costruire reti logiche combinatorie e sequenziali collegando graficamente porte logiche, FF, contatori, mux/demux, memorie e altri componenti fondamentali.
 - L'ambiente è organizzato per livelli di astrazione (sotto-circuiti e progetti complessi attraverso progettazione gerarchica).
 - Consente di osservare segnali logici in tempo reale, di inserire clock e analizzare la variazione dei livelli logici in una simulazione interattiva.
 - E' **open-source** e multiplatforma → si può *installare su qualsiasi sistema operativo*.
 - Non supporta linguaggi di descrizione hardware (HDL) e non permette la simulazione di microcontrollori/processori complessi (che non sono obiettivo del corso).



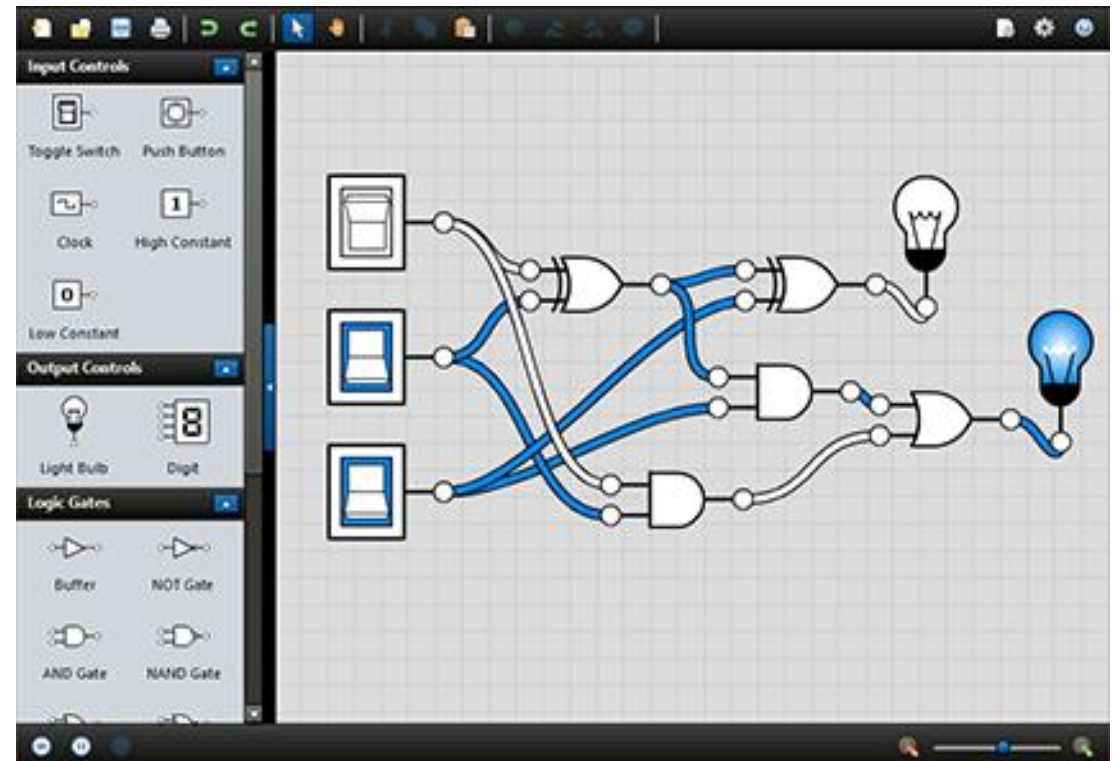
PRINCIPALI SIMULATORI DI CIRCUITI DIGITALI: DIGITAL

- **Digital** (<https://github.com/hneemann/Digital>) è un moderno simulatore **open-source** sviluppato per finalità didattiche, che combina un'interfaccia di facile comprensione con funzionalità di analisi temporale. A differenza di Logisim, permette di:
 - Visualizzare le forme d'onda temporali (**waveforms**) durante la simulazione, per osservare in modo più preciso i segnali ed i ritardi introdotti nei circuiti.
 - Studio dei circuiti sequenziali, delle macchine a stati finiti (**FSM**) e delle reti sincrone governate dal clock.
- Include componenti base come porte logiche, mux, demux, contatori, registri e componenti più complessi come ALU e memorie RAM/ROM.
- Meno adatto per simulazioni elettroniche complete, a causa di componenti analogici e microcontrollori.

Questo è il simulatore che utilizzeremo durante il corso!

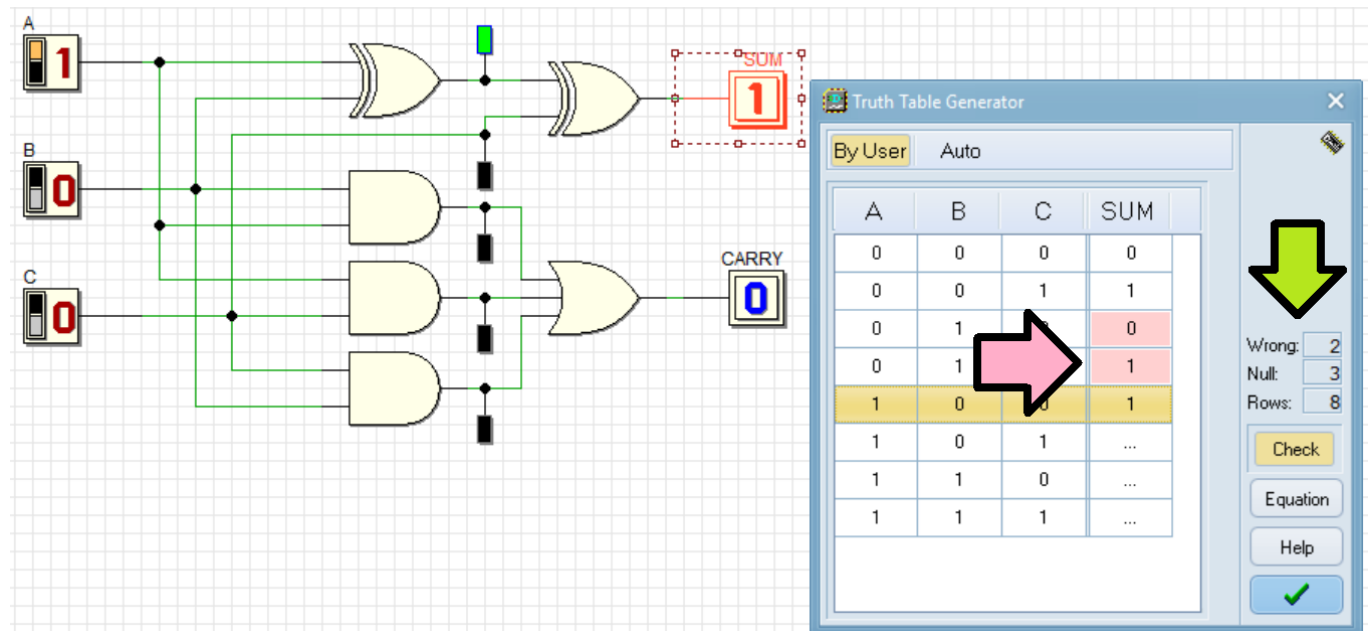
PRINCIPALI SIMULATORI DI CIRCUITI DIGITALI: LOGIC.LY

- **Logic.ly** (<https://logic.ly/>) è un simulatore di logica digitale. È un software **commerciale**, disponibile in versione web e desktop, ed una versione gratuita per sperimentare un numero limitato di componenti.
 - Interfaccia grafica intuitiva e accessibile
 - Permette di attivare in tempo reale gli ingressi e visualizzare le variazioni delle uscite
 - Facilita la comprensione delle tabelle di verità e del principio di propagazione dei segnali.
 - Versione gratuita limitata, non consente la realizzazione di reti complesse o l'uso di elementi avanzati come memorie.
 - Non è pensato per progetti gerarchici o circuiti su più livelli.



PRINCIPALI SIMULATORI DI CIRCUITI DIGITALI: DEEDS

- **Deeds** (<https://www.digitalelectronicsdeeds.com/>), acronimo di Digital Electronics Education and Design Suite, è un ambiente per la progettazione e simulazione di circuiti digitali. Integra tre strumenti principali:
 1. DLS (Deeds Logic Simulator) è il simulatore **logico**
 2. DMC (Deeds Microcomputer Simulator) è il simulatore di sistemi digitali **programmabili**
 3. DSS (Deeds Schematic Simulator) è il simulatore di circuiti logici descritti in **VHDL** (linguaggio di descrizione hardware).
- Il modulo DLS permette di costruire e testare circuiti con porte logiche, flip-flop, registri, contatori, ALU e memorie, visualizzando in tempo reale i segnali logici e le transizioni temporali.
- È un **software free** per scopi educativi, ma disponibile **solo per Windows**.





SIMULATORE DIGITAL

DOWNLOAD ED INSTALLAZIONE

- Andare su: <https://github.com/hneemann/Digital>
 - JRE (Java Runtime Environment) ≥ 8** è richiesto per poter funzionare.

1. Cliccare su download

Download

2. Decomprimere la cartella *Digital.zip* che viene scaricata e posizionarla su Desktop o dove opportuno. Dovremo lavorare **all'interno di questa cartella!**

Digital

Digital is an easy-to-use digital logic designer and circuit simulator

Digital

Nome
Digital_noD3D.exe
Digital.exe
Digital.jar
Digital.sh
> docu
> examples
icon.svg
install.sh
> lib
> linux
ReleaseNotes.txt
Version.txt
Windows_Read.me

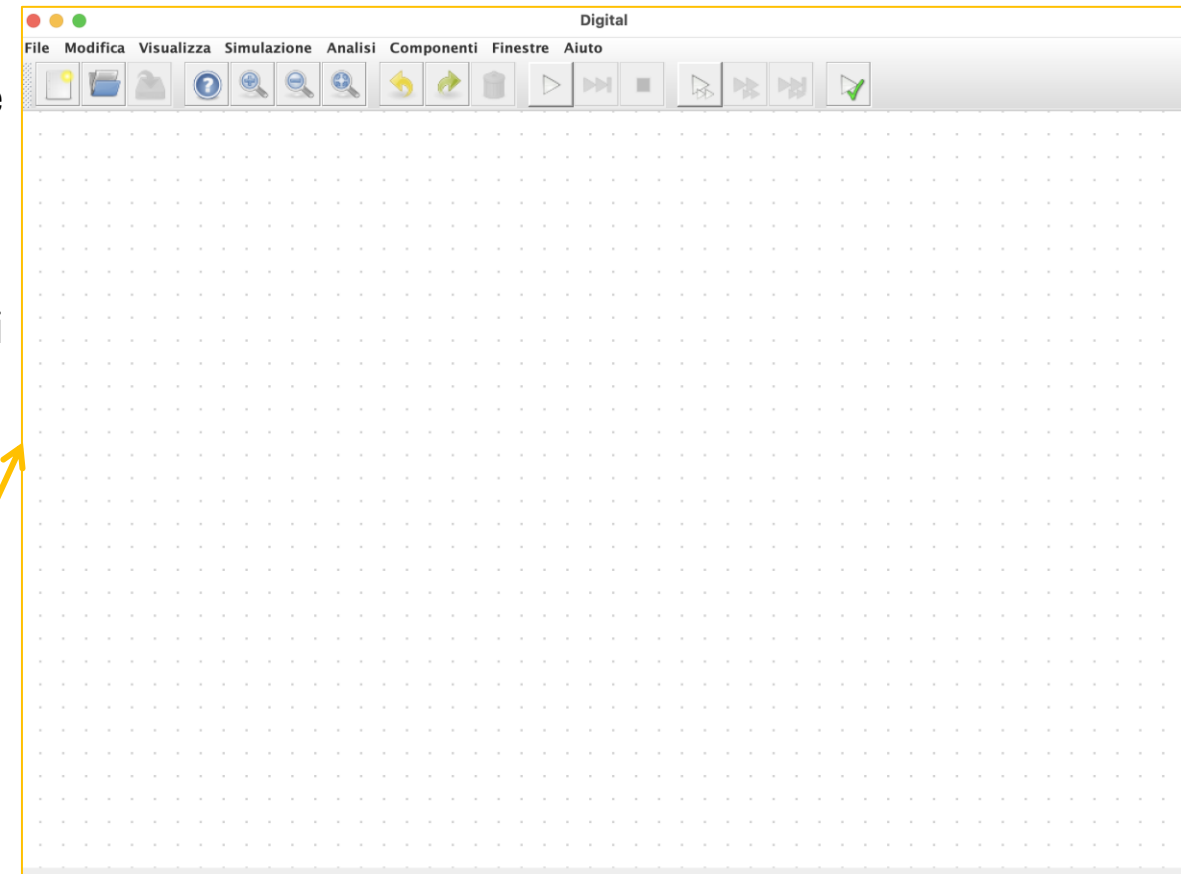
AVVIO

- Per avviare il simulatore:
 - Per Windows e MacOS, basta cliccare il file .jar presente nella cartella e il simulatore si apre automaticamente.
 - Per Linux (ma anche per altri OS), aprire il terminale e dare il comando:

`java -jar Digital.jar`

N.B. per lavorare da linea di comando occorre posizionarsi nella cartella in cui è stato scaricato il software.

```
alessandrasomma@MacBook-Pro-di-Alessandra Digital % java -jar Digital.jar
```



MODALITA' DI FUNZIONAMENTO

Il simulatore ha **due modalità di funzionamento**: modifica e simulazione.

- In **modifica**, si possono apportare modifiche al circuito. Gli utenti possono aggiungere o connettere componenti. In questa modalità, *la simulazione è disabilitata*.

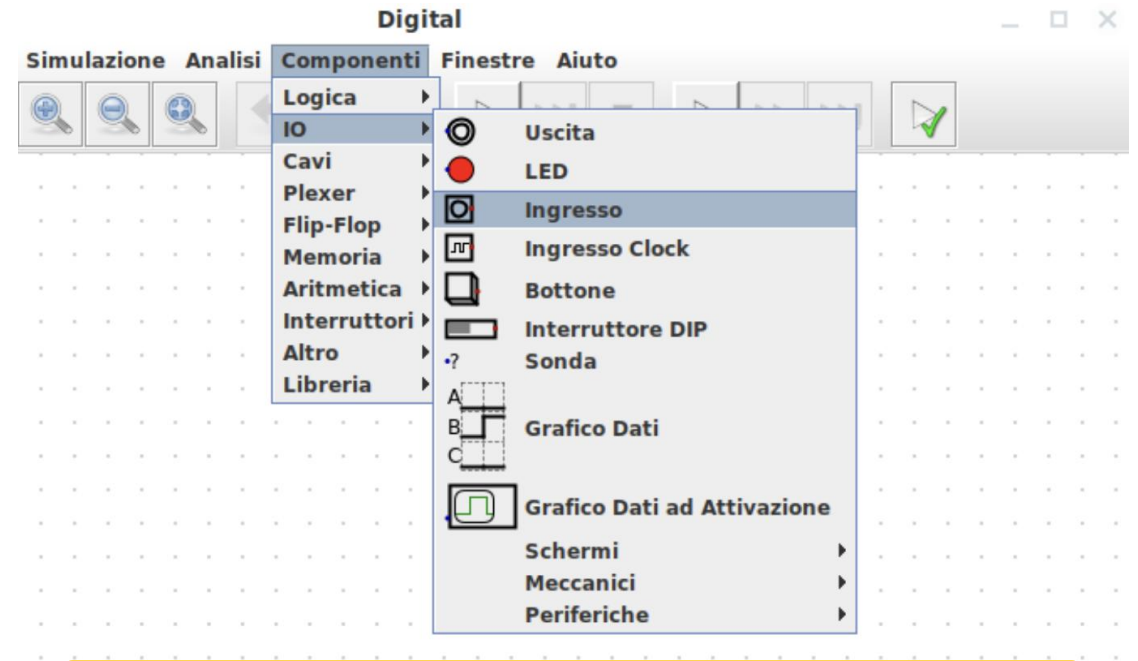


- In **simulazione**, si avvia la simulazione premendo il tasto *Avvia* nella barra degli strumenti. Se ci sono errori nel circuito, un messaggio appropriato è mostrato e i componenti o cavi affetti da problemi sono evidenziati.
 - Se il circuito non ha errori, la simulazione è abilitata. Ora si può interagire con l'esecuzione della simulazione.
 - Nella modalità simulazione *non è possibile modificare il circuito*. Per far ciò, bisogna attivare la modalità di modifica ancora, fermando prima la simulazione.

COMPONENTI

- Nel menu “Componenti” sono raccolte tutte le categorie di elementi disponibili. Le voci principali sono:

- **Logica:** porte logiche fondamentali come AND, OR NOT etc per costruire circuiti che implementano funzioni booleane e macchine combinatorie.
- **IO:** elementi che permettono interazione tra utente e circuito, e visualizzazione del risultato (e.g. interruttori, pulsanti, grafici dati)
- **Cavi:** include i fili, i «bus», i connettori, utili per organizzare i collegamenti tra componenti.
- **Plexer:** componenti specializzati come mux, demux, che permettono selezione/instradamento di segnali.
- **Flip-Flop:** elementi sequenziali che contengono stato interno (FF RS, D, JK, T, registri, contatori).
- **Memoria:** ROM, RAM, registri, moduli di memoria parametrizzati.
- **Aritmetica:** addizionatori, sottrattori, comparatori, ALU, etc.
- Interruttori/Altro/Libreria

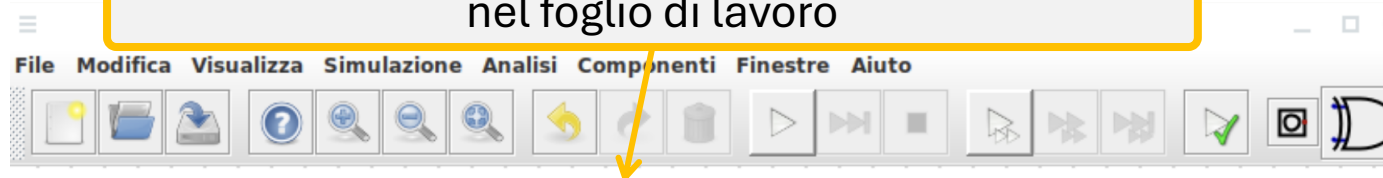


Questa sezione è fondamentale per creare i punti di ingresso e di uscita dei nostri circuiti.

PRIMO ESEMPIO IN DIGITAL: CREAZIONE

- Obiettivo: Disegnare **porte logiche elementari** (per esempio la XOR) e vedere la tabella di verità.

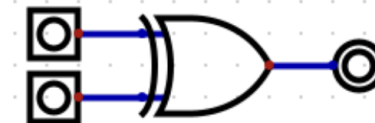
1. Inserire i componenti (ingresso, uscita opportuni nel foglio di lavoro



porta
logica
XOR



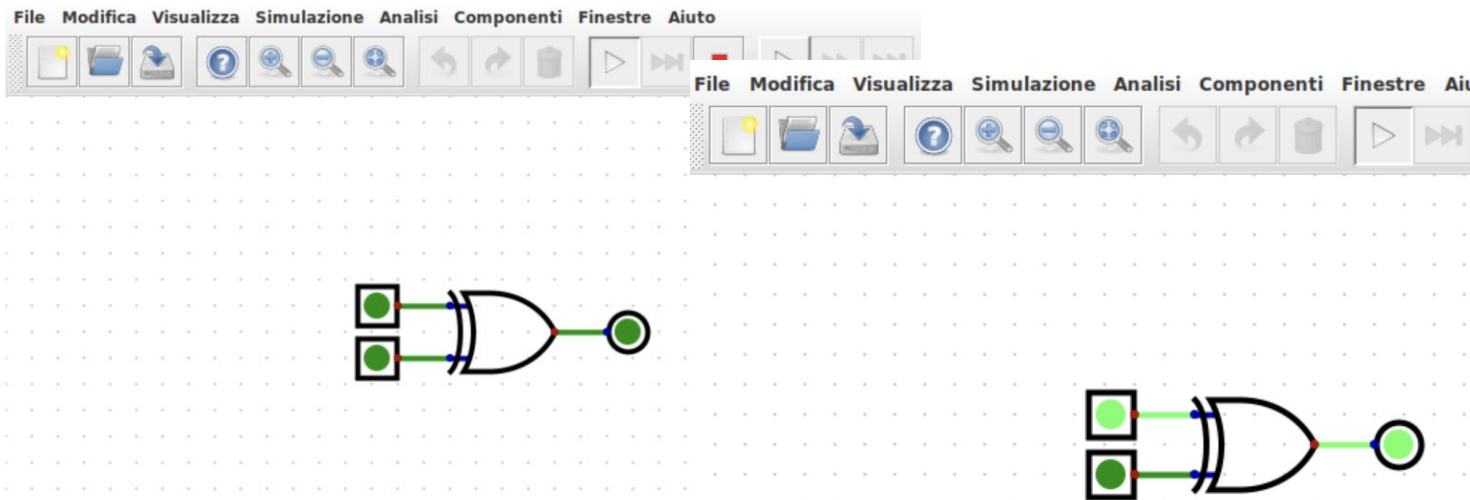
2. Collegare i componenti con i cavi.



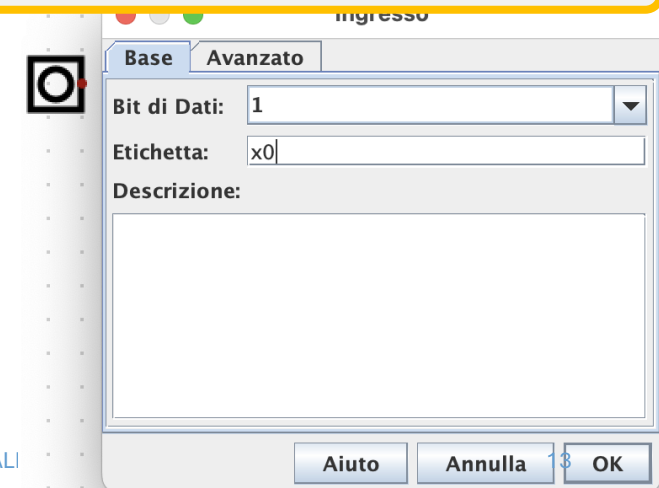
utilizza il mouse per creare una connessione tra i pallini blu e rosso. Assicurati che esattamente un pallino rosso è connesso ad un certo numero di pallini blu.

PRIMO ESEMPIO IN DIGITAL: SIMULAZIONE

- L'interazione con il circuito è possibile quando la simulazione è stata avviata. Questo si effettua premendo il bottone di **avvio**, nella barra degli strumenti.
- Dopo aver avviato la simulazione, il colore dei cavi cambia e gli ingressi ed uscite sono ora calcolati:
 - verde chiaro: indica un valore logico "1",
 - verde scuro indica un valore logico "0"

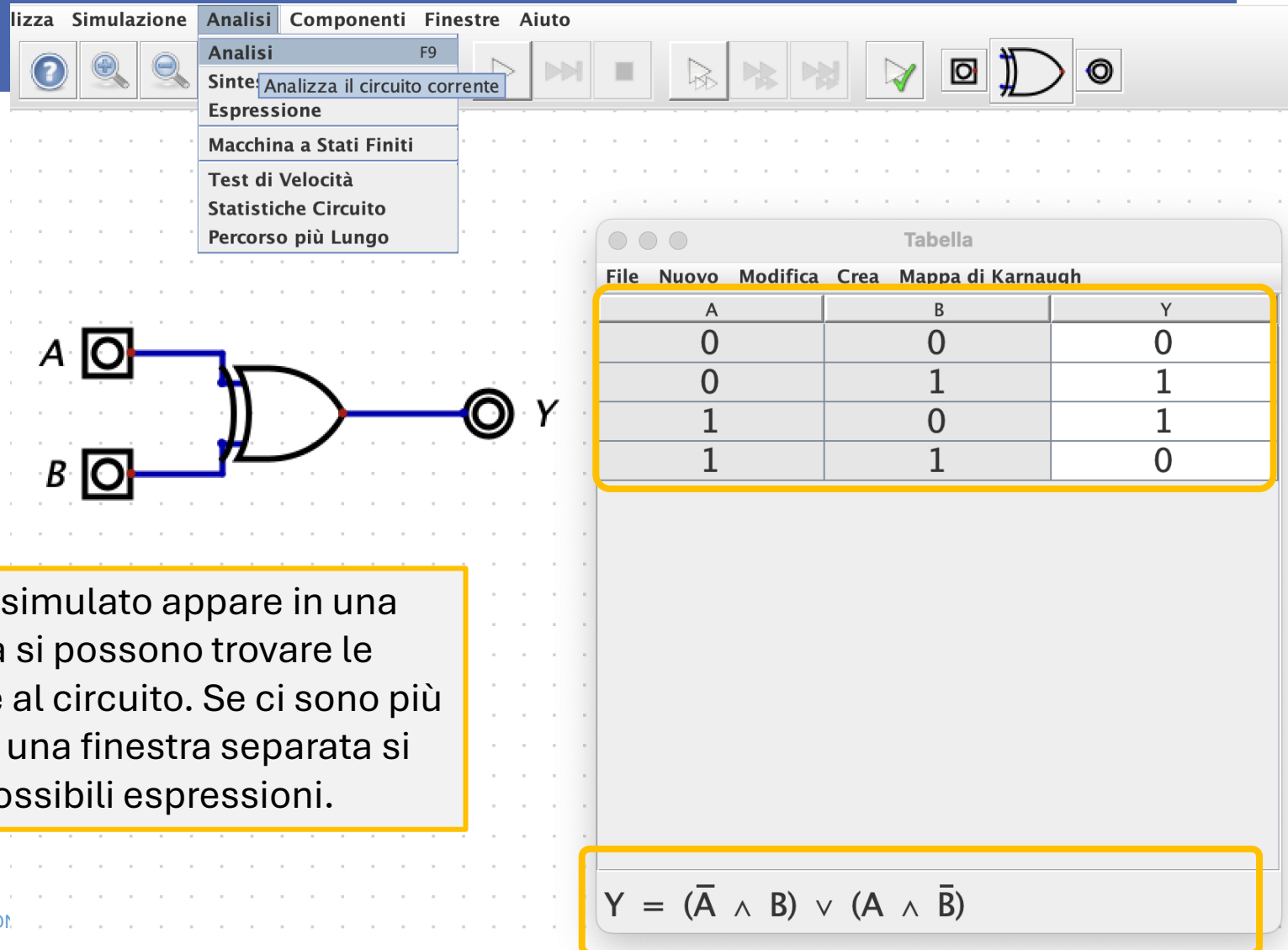


Tornando in *modalità modifica* è possibile dare un nome (etichetta) ad ingressi ed uscite con tasto dx sullo specifico componente.



PRIMO ESEMPIO IN DIGITAL: ANALISI

- L'opzione nel menù *Analisi* contiene una funzionalità di **Analisi**. Questa funzione effettua un'analisi del circuito attuale (SOLO se tutti gli ingressi ed uscite sono stati etichettati appropriatamente).



Analizza Simulazione **Analisi** Componenti Finestre Aiuto

Analisi F9

Sintesi Analizza il circuito corrente

Espressione

Macchina a Stati Finiti

Test di Velocità

Statistiche Circuito

Percorso più Lungo

A B Y

Tabella

File Nuovo Modifica Crea Mappa di Karnaugh

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$Y = (\bar{A} \wedge B) \vee (A \wedge \bar{B})$$

La **tabella di verità** del circuito simulato appare in una nuova finestra. Sotto la tabella si possono trovare le **espressioni algebriche** associate al circuito. Se ci sono più possibili espressioni algebriche, una finestra separata si aprirà, mostrando tutte le possibili espressioni.

PRIMO ESEMPIO IN DIGITAL: ANALISI

La finestra di dialogo contenente la tabella ha una opzione nel menù principale chiamata **Mappa di Karnaugh**, che consente la visualizzazione della tabella di verità con una mappa.

Nella parte superiore c'è una lista che consente la **selezione dell'espressione desiderata** da visualizzare nella mappa di Karnaugh. In questo modo è possibile, per esempio, illustrare come alcune espressioni algebriche equivalenti possono risultare (in questo esempio però c'è una sola espressione minimizzata)

Mappa di Karnaugh

$Y = (\bar{A} \wedge B) \vee (A \wedge \bar{B})$

	$\bar{B}$	B	
$\bar{A}$	0	1	0
A	1	0	0

0 1 2 3

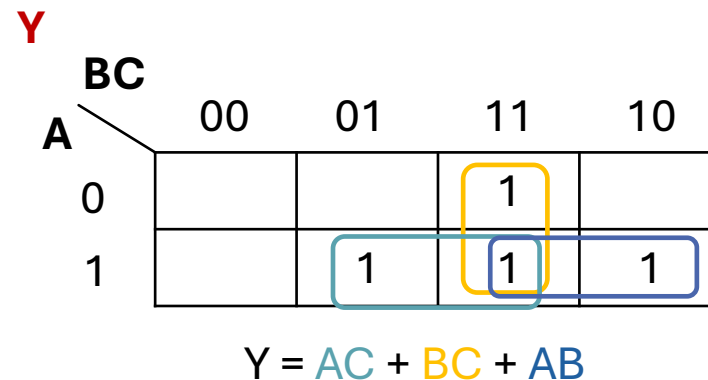
ESERCIZIO - VOTO A MAGGIORANZA: CREAZIONE

- Obiettivo: Progettare una macchina combinatoria a tre ingressi (A,B,C) tale per cui l'uscita Y è alta se almeno due dei suoi ingressi sono alti.

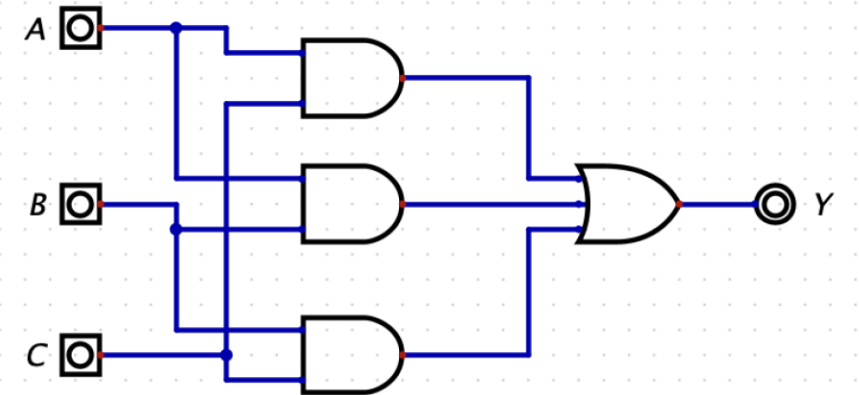
Tabella verità

A	B	C	y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

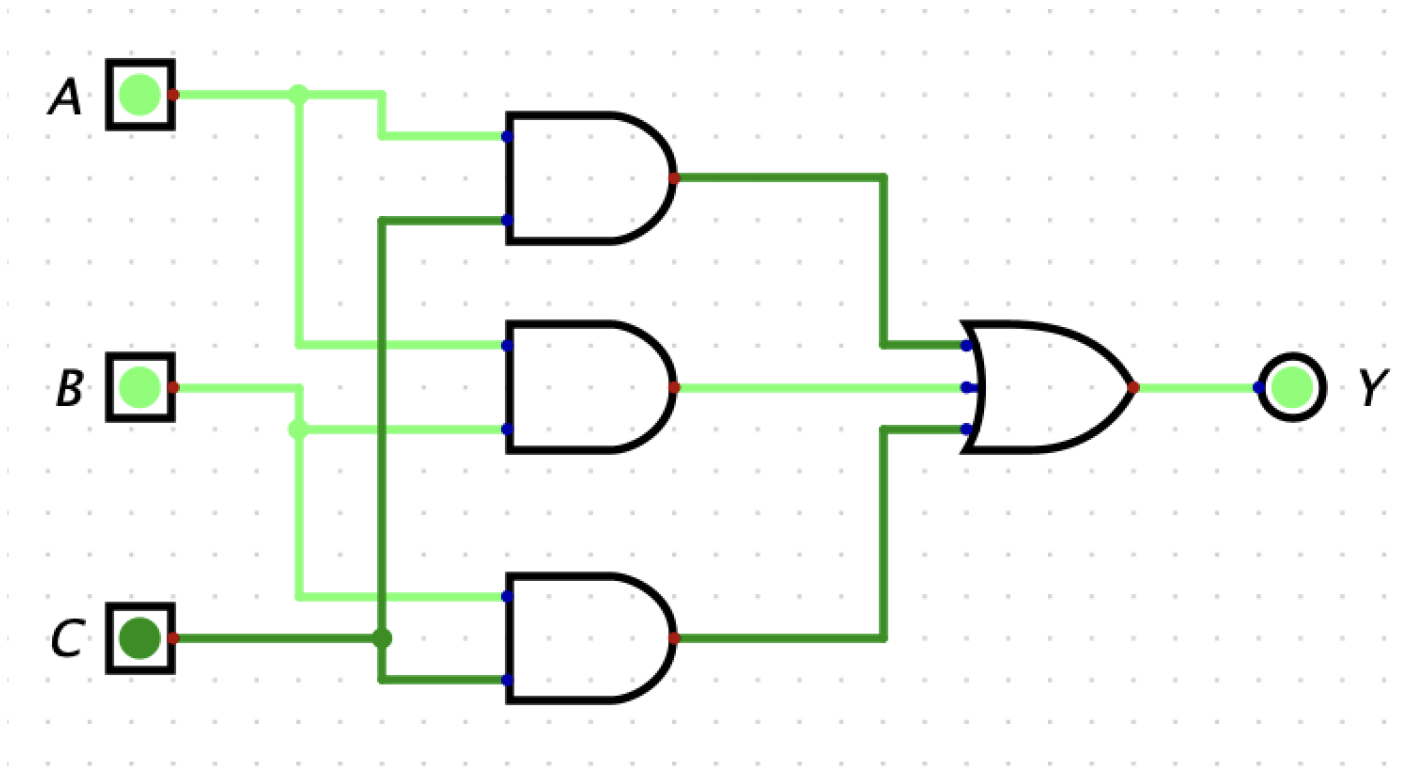
Ottimizzazione con Karnaugh



Circuito in Digital



ESERCIZIO - VOTO A MAGGIORANZA: SIMULAZIONE



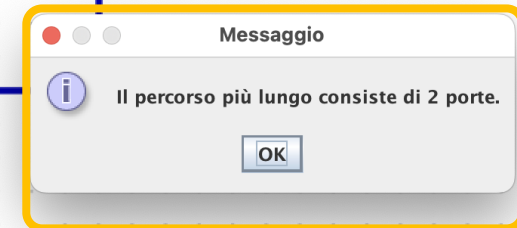
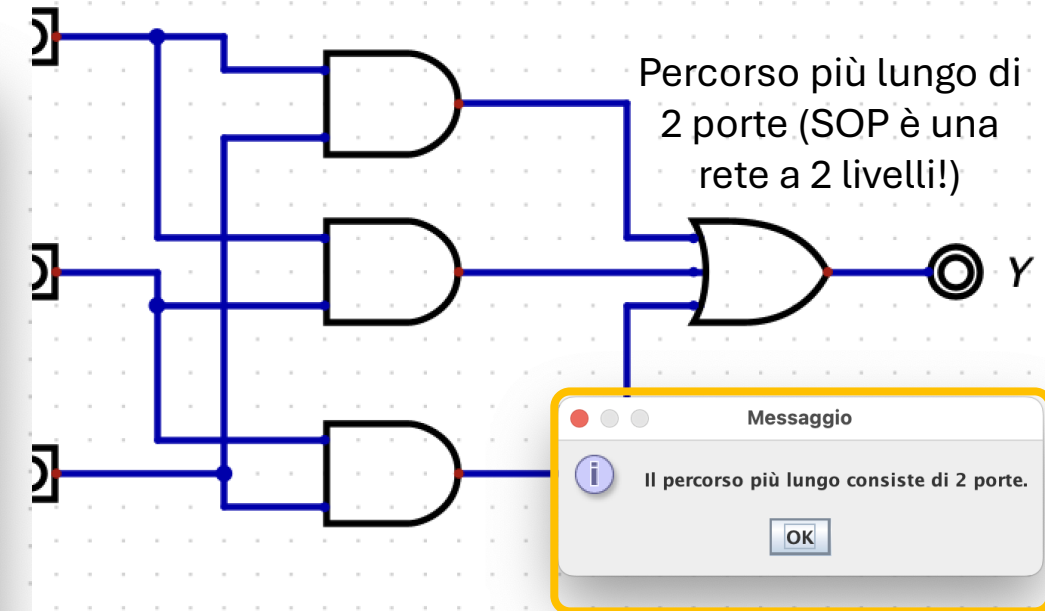
Avviando la simulazione possiamo manipolare gli input per studiare il comportamento dell'uscita Y. Ad esempio se poniamo A e B ad 1, Y diventa effettivamente alta.

ESERCIZIO - VOTO A MAGGIORANZA: ANALISI

Tabella				
File	Nuovo	Modifica	Crea	Mappa di Karnaugh
A	B	C	Y	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	
1	0	0	0	
1	0	1	1	
1	1	0	1	
1	1	1	1	

$Y = (A \wedge C) \vee (A \wedge B) \vee (B \wedge C)$

Mappa di Karnaugh				
$Y = (A \wedge C) \vee (A \wedge B) \vee (B \wedge C)$				
	$\bar{B}$		B	
$\bar{A}$	0	0	1	0
A	0	1	1	1
	$\bar{C}$		C	



Statistiche Circuito				
Componenti	Ingressi	Bit	Bit di Indirizzo	Numeri
And	2	1		3
Or	3	1		1

ESERCIZIO – DECODER 2:4

- Obiettivo: Decodificatore 2:4

Tabella verità

A	B	Y3	Y2	Y1	Y0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

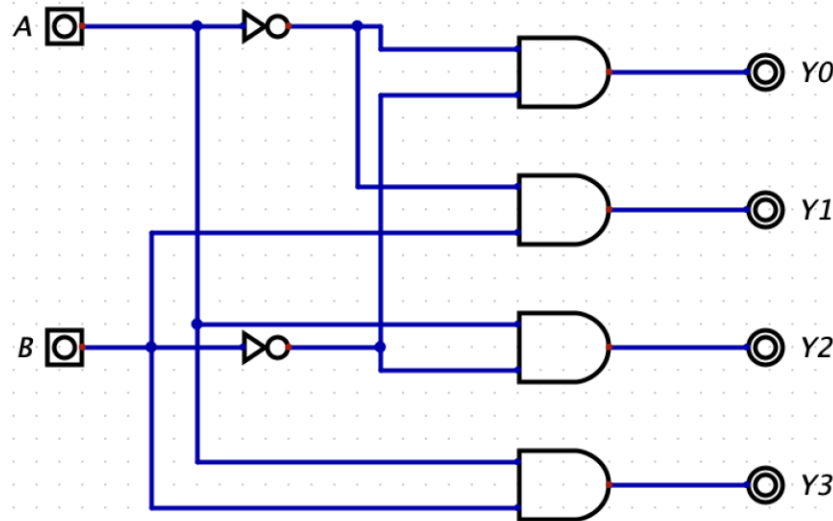
$$Y0 = A'B'$$

$$Y1 = A'B$$

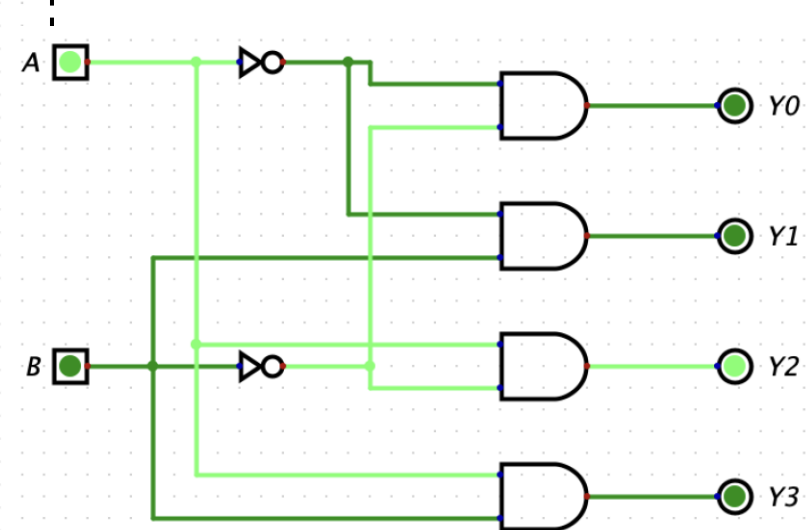
$$Y2 = AB'$$

$$Y3 = AB$$

Circuito in Digital

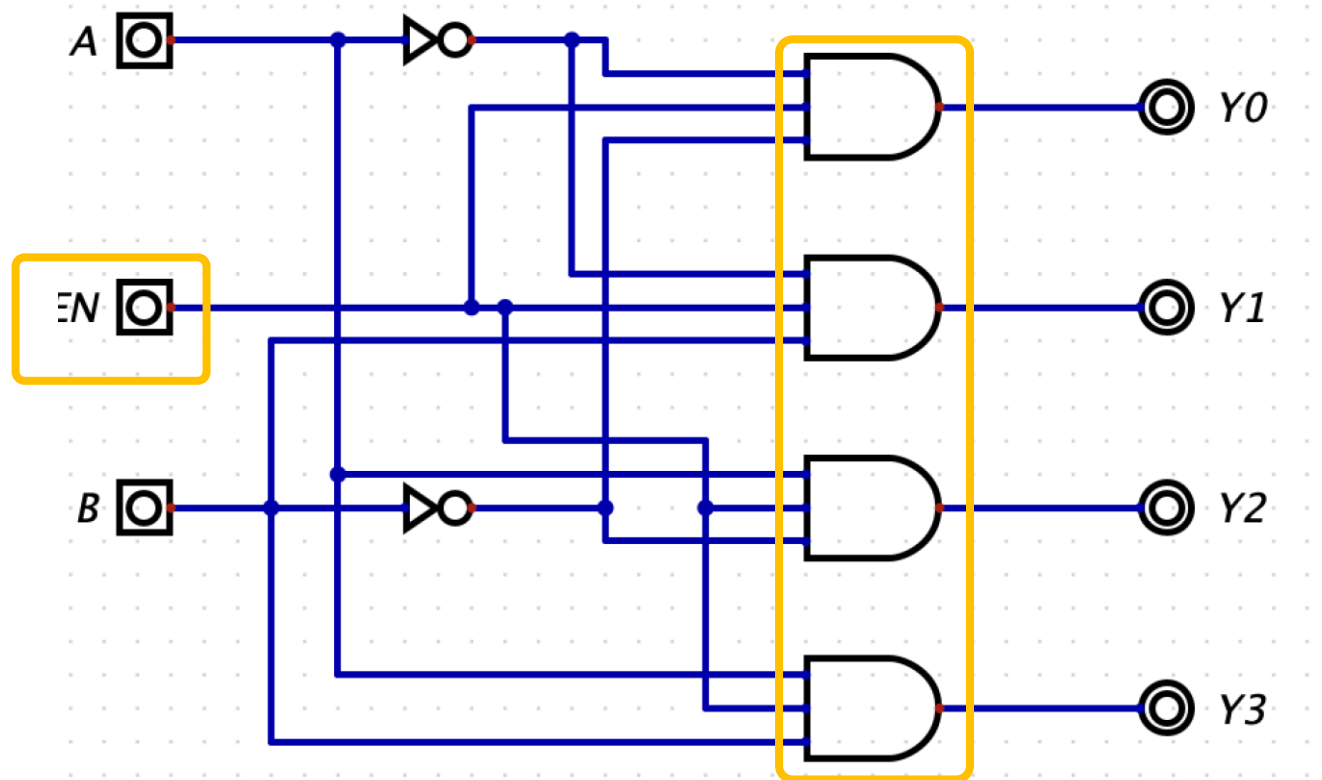


Simulazione



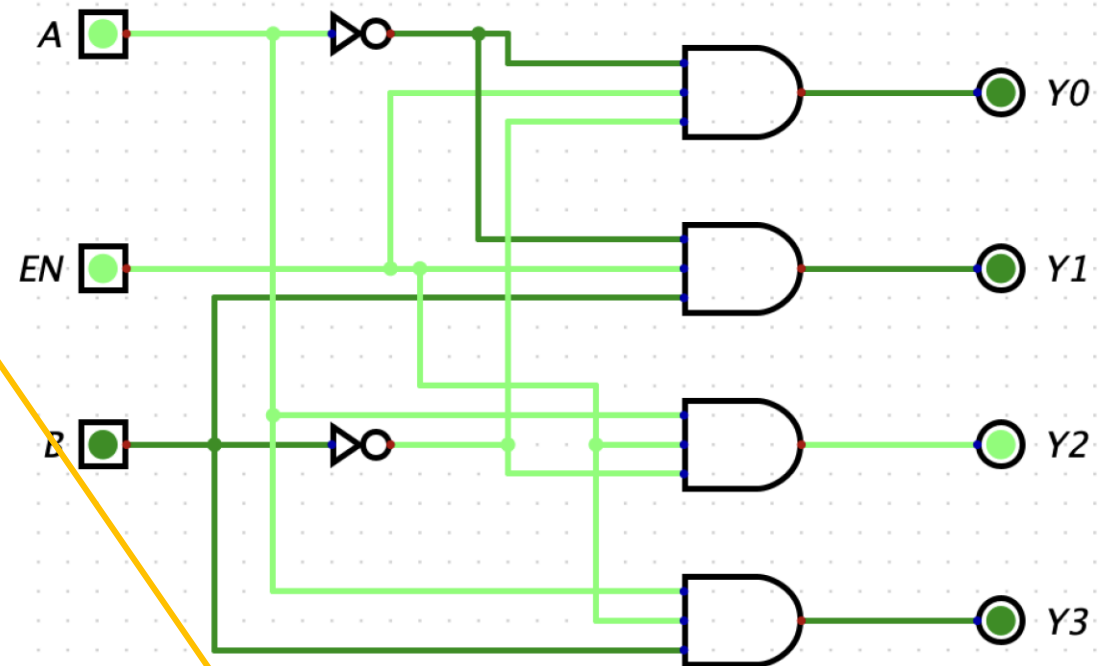
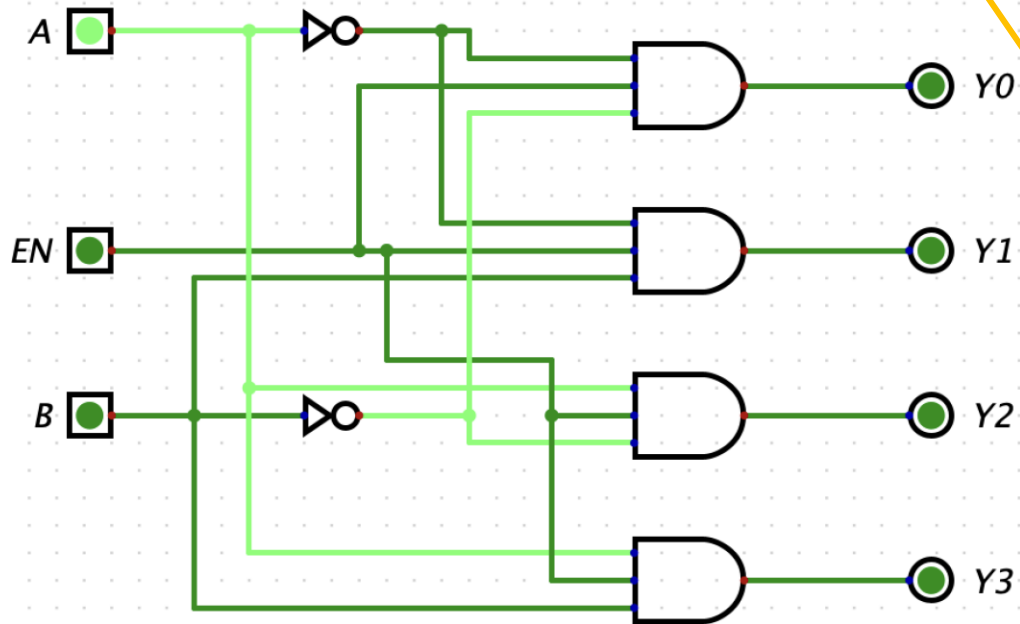
ESERCIZIO – DECODER 2:4 CON ENABLE

- Possiamo aggiungere un **segnale di enable EN** che abiliti o meno il circuito di decodifica. Questo vuol dire che, oltre ad aggiungere l'ingresso EN, il segnale EN deve andare in ingresso ad ogni AND (che da 2 passa a 3 ingressi).



ESERCIZIO – DECODER 2:4 CON ENABLE

Anche se $AB = 01$, l'uscita $Y2 = 0$,
perché $EN = 0$



Se $EN = 1$, per $AB = 10$, l'uscita $Y2 = 1$

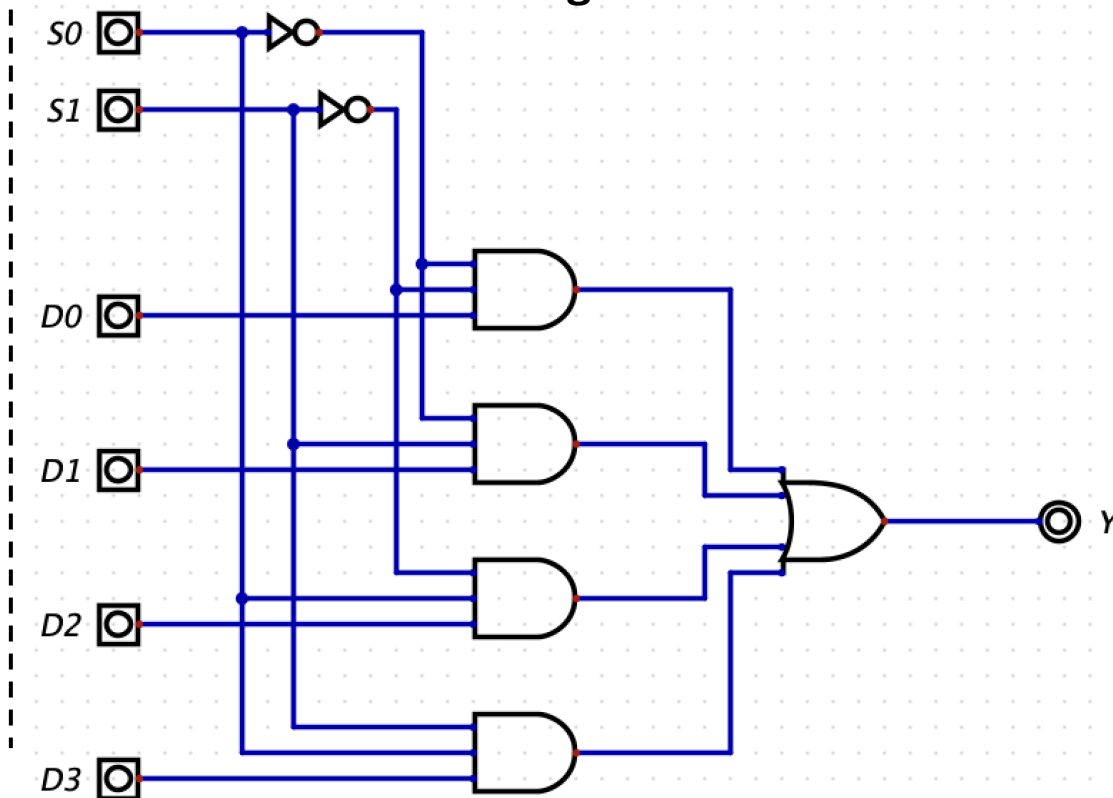
ESERCIZIO – MULTIPLEXER 4:1 INDIRIZZABILE

- Obiettivo: Realizzare un mux 4:1 indirizzabile. Dati 4 ingressi, D0-D3, e due segnali di selezione S0-S1:

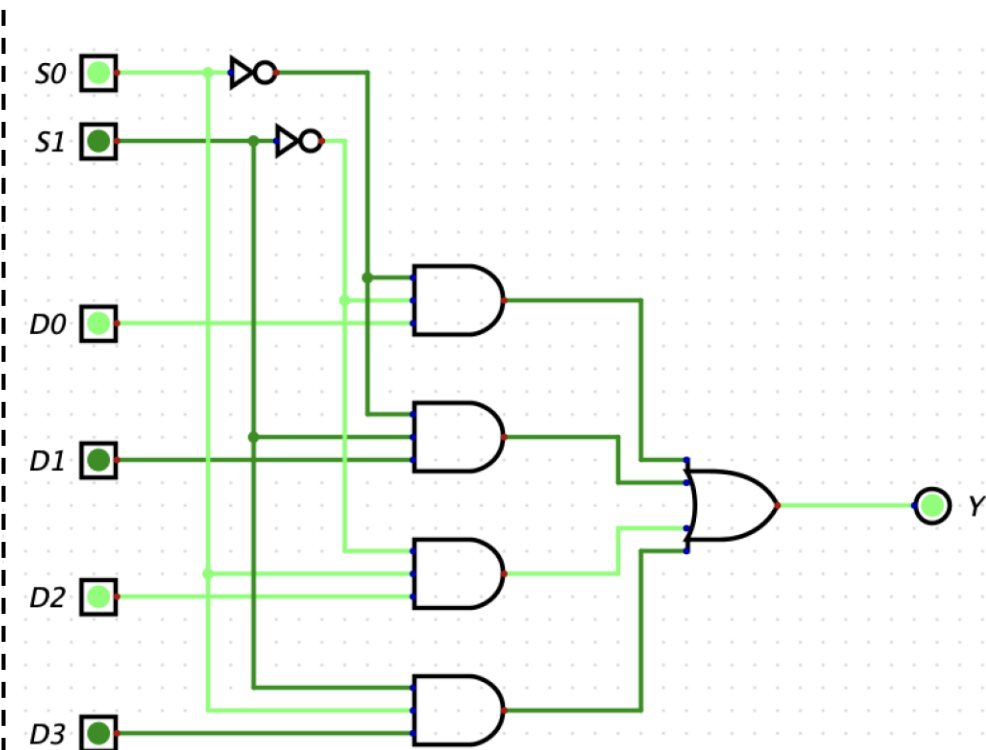
Tabella verità

S0	S1	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

Circuito in Digital

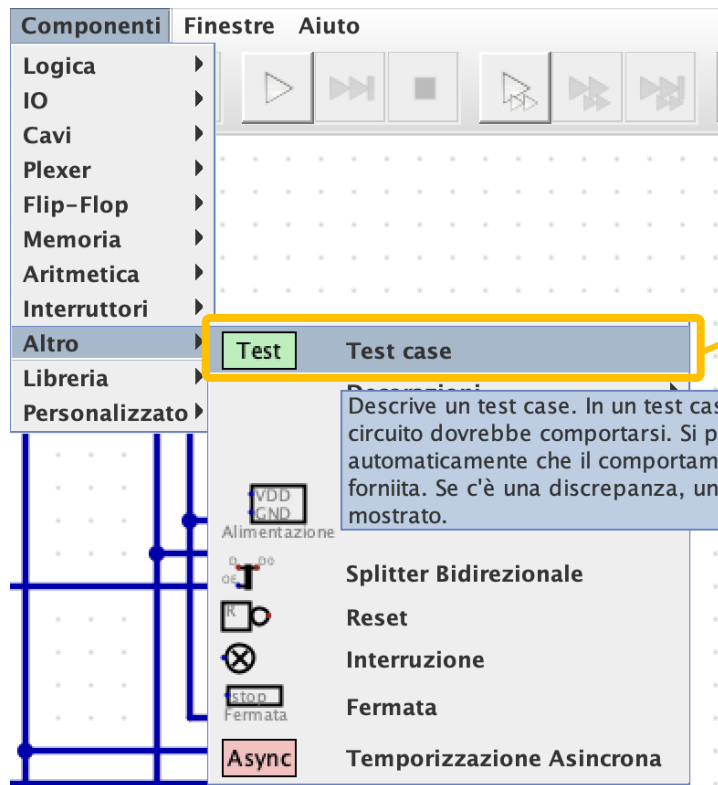


Simulazione



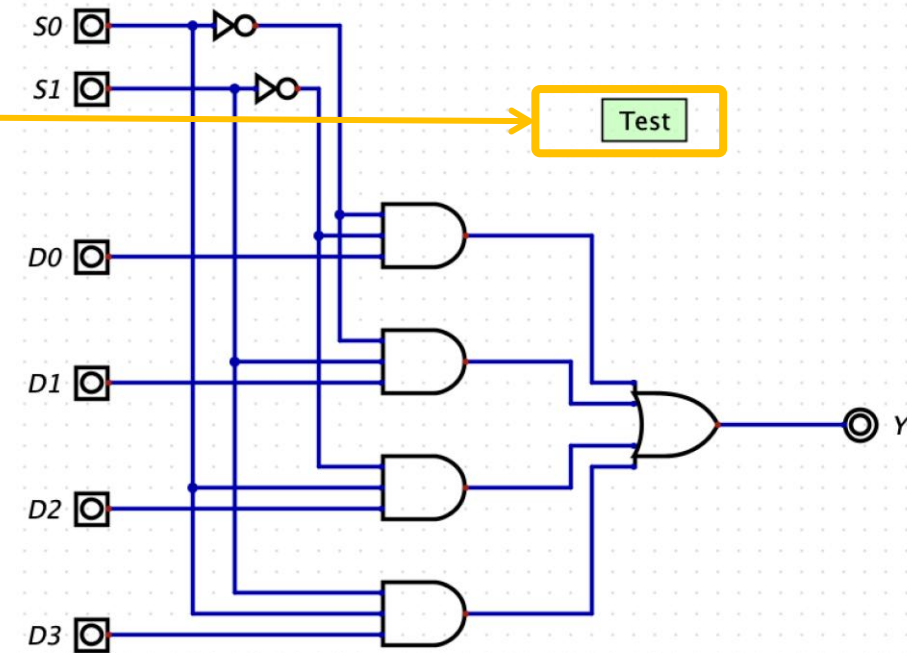
ESERCIZIO – MULTIPLEXER 4:1 INDIRIZZABILE: TEST CASE

- **Test case:** Per verificare la corretta funzionalità del multiplexer, **un test case** dovrebbe essere aggiunto. In questo caso specifichiamo la tabella di verità, che dovrebbe rispecchiare quella del circuito. In questo modo è automaticamente possibile verificare la corrispondenza.



Dati di Test

	S0	S1	D3	D2	D1	D0	Y
1							
2	0	0	X	X	X	0	0
3	0	0	X	X	X	1	1
4	0	1	X	X	0	X	0
5	0	1	X	X	1	X	1
6	1	0	X	0	X	X	0
7	1	0	X	1	X	X	1
8	1	1	0	X	X	X	0
9	1	1	1	X	X	X	1



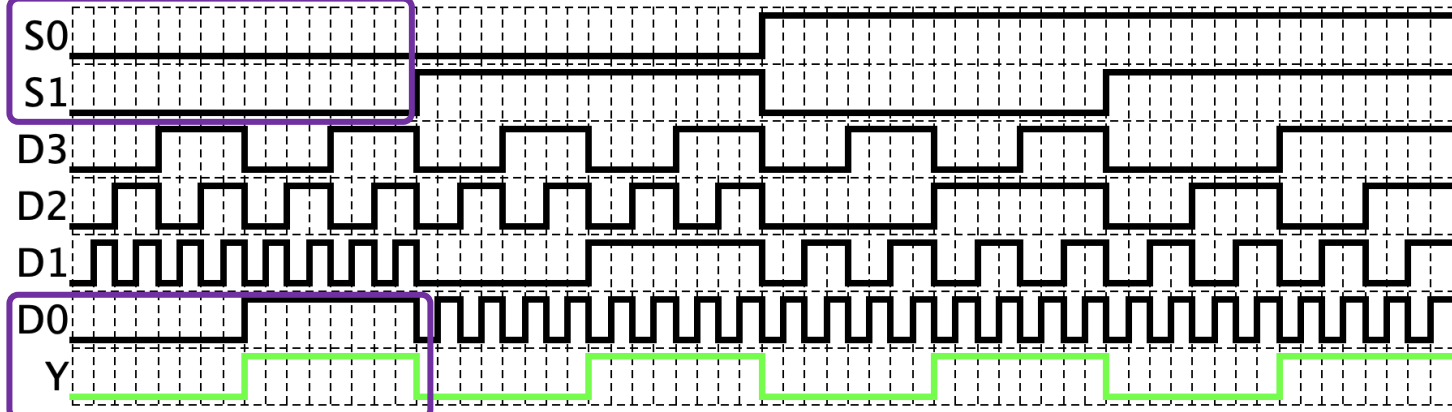
ESERCIZIO – MULTIPLEXER 4:1 INDIRIZZABILE: TEST CASE

- Per eseguire il test case, bisogna andare in simulazione e cliccare *Esegui tutti i test*:

passati	S0	S1	D3	D2	D1	D0	Y
L2;X=0	0	0	0	0	0	0	0
L2;X=1	0	0	0	0	1	0	0
L2;X=2	0	0	0	1	0	0	0
L2;X=3	0	0	0	1	1	0	0
L2;X=4	0	0	1	0	0	0	0
L2;X=5	0	0	1	0	1	0	0
L2;X=6	0	0	1	1	0	0	0
L2;X=7	0	0	1	1	1	0	0
L3;X=0	0	0	0	0	0	1	1
L3;X=1	0	0	0	0	1	1	1
L3;X=2	0	0	0	1	0	1	1
L3;X=3	0	0	0	1	1	1	1
L3;X=4	0	0	1	0	0	1	1
L3;X=5	0	0	1	0	1	1	1
L3;X=6	0	0	1	1	0	1	1
L3;X=7	0	0	1	1	1	1	1

Ad esempio per $S0S1=00$, sappiamo che il mux 4:1 deve selezionare $D0$, infatti anche se $D3, D2, D1$ variano, $Y=D0$.

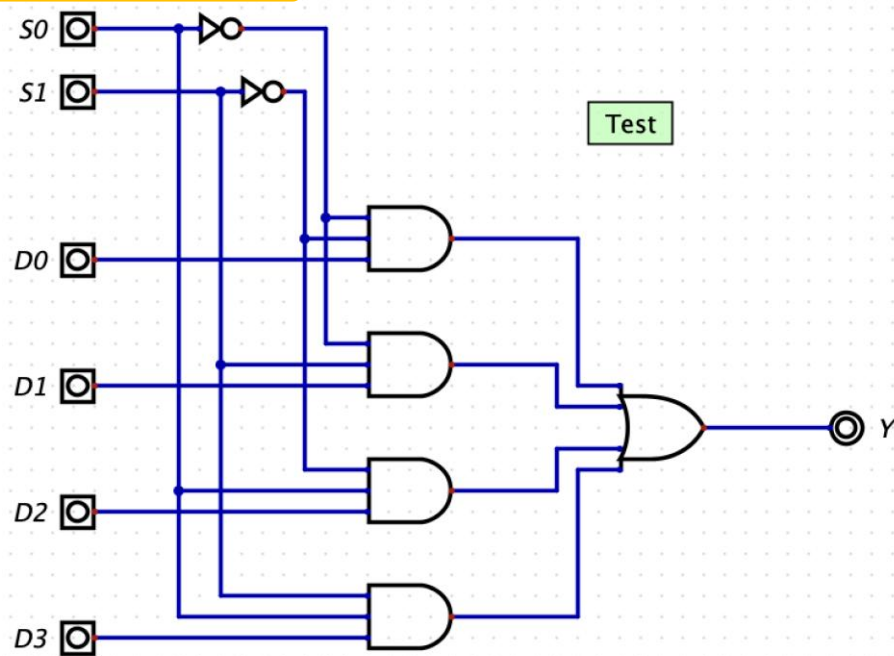
Possiamo anche visualizzare il risultato sotto forma di grafico



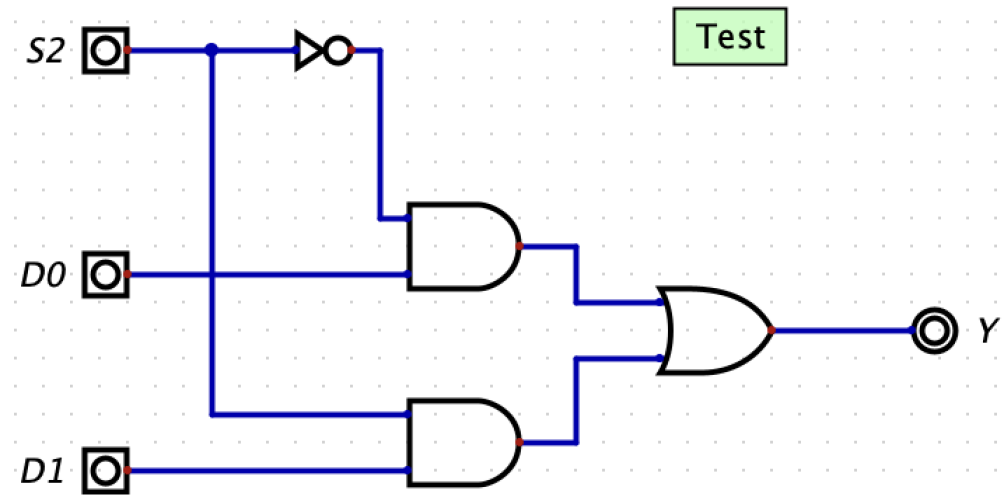
ESERCIZIO – COMPOSIZIONE DI MULTIPLEXER

- Obiettivo: Vogliamo realizzare un MUX 8:1 a partire da MUX 4:1 e MUX 2:1 **per composizione**.

File *mux41.dig*

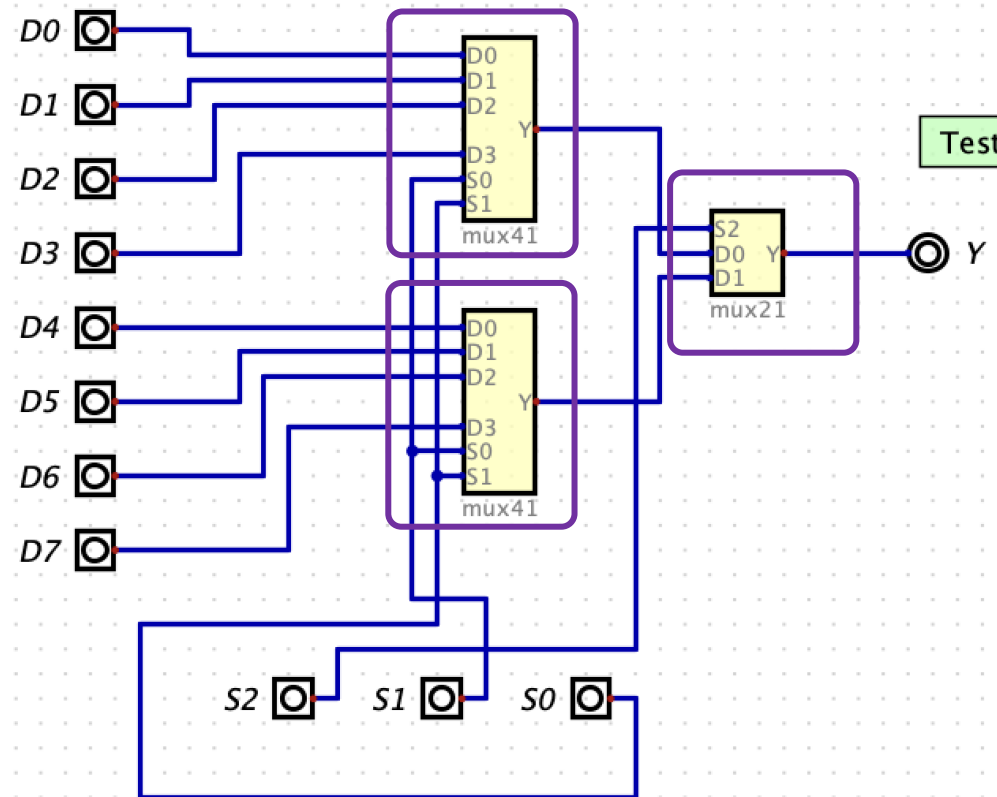
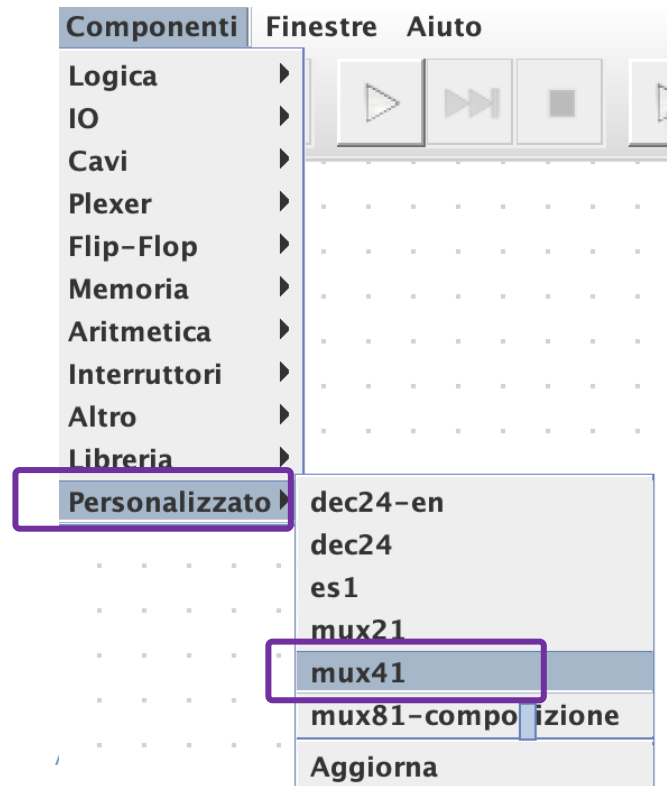


File *mux21.dig*



ESERCIZIO – COMPOSIZIONE DI MULTIPLEXER

- Obiettivo: Vogliamo realizzare un MUX 8:1 a partire da MUX 4:1 e MUX 2:1 **per composizione**.
- Creiamo un nuovo file mux81.dig (n.b. va salvato nella stessa cartella dove sono i file mux41.dig e mux21.dig) ed **importiamo i mux 4:1 e 2:1**



S2S1S0 sono posizionati da MSB a LSB, da notare che S0 ed S1 del mux 8:1 sono collegati a S1-S0 del mux 4:1 (per come sono stati indicati nel mux 4:1)

ESERCIZIO – COMPOSIZIONE DI MULTIPLEXER

- Per esser sicuri che il comportamento **complessivo** del MUX 8:1 sia corretto scriviamo un test case.

Dati di Test

	S2	S1	S0	D7	D6	D5	D4	D3	D2	D1	D0	Y
1	1	0	0	X	X	X	0	X	X	X	X	0
3	1	0	0	X	X	X	1	X	X	X	X	1

passati

	S2	S1	S0	D7	D6	D5	D4	D3	D2	D1	D0	Y
L2;...	1	0	0	0	0	0	0	0	0	0	0	0
L2;...	1	0	0	0	0	0	0	0	0	0	1	0
L2;...	1	0	0	0	0	0	0	0	0	1	0	0
L2;...	1	0	0	0	0	0	0	0	0	1	1	0
L2;...	1	0	0	0	0	0	0	0	1	0	0	0
L2;...	1	0	0	0	0	0	0	0	1	0	1	0

